Learning Guide

Libft

Technical Guide and Implementation Reference

Environment, C Fundamentals, and 43 Functions

A guide to understanding, implementing, and testing the functions of the Libft library.

Common Core — 1337

Working document and technical reference

Preface

This guide accompanies the Libft project. It brings together the compilation rules, the C concepts used throughout the project, and a practical reference for the functions to implement.

The guide follows three steps: understand the project environment and constraints, connect function families to fundamental C concepts, then consult prototypes, expected behavior, and edge cases during development and testing.

> *The goal is not merely to reproduce a function's output, but to understand its preconditions, the memory it manipulates, and the limits of its behavior.*

The examples are learning aids: they do not replace the official project specification, school instructions, or the 42 Norm. If they differ, the project and evaluation requirements take precedence.

Contents

Preface **i**

1 Technical Environment, Compilation, and Project Rules **1**

1.1 The Header File (`libft.h`) 1

1.2 Automating Compilation with a `Makefile` 2

1.3 Compiling and Creating a Static Library 2

1.4 Coding Rules, Scope, and Memory Management 3

2 Function Classification and Fundamental C Concepts **5**

2.1 Function Families 5

2.2 Fundamental Concepts 6

2.3 Summary — Concepts to Master 16

3 Complete Reference to the 43 Libft Functions **17**

3.1 Part 1 — Libc Functions (23 Functions) 17

3.2 Part 2 — Additional Functions (11 Functions) 20

3.3 Part 3 — Linked Lists (9 Bonus Functions) 22

3.4 Final Validation Checklist 23

1 Technical Environment, Compilation, and Project Rules

1.1 The Header File (`libft.h`)

1.1.1 Purpose of the Header File

In C, to use a function defined in another file, the compiler must know its **prototype**: its name, return type, and parameters.

The `libft.h` file centralizes the elements shared across our library:

- function prototypes (`ft_strlen`, `ft_memcpy`, etc.);
- data structures, especially `t_list` for the bonus functions;
- required includes (`<stdlib.h>`, `<unistd.h>`, etc.).

Each source file can access these declarations with:

```
#include "libft.h"
```

1.1.2 Include Guards (*Header Guards*)

A header file can be included multiple times, directly or indirectly, in a source file. This can cause redefinition errors.

Include guards prevent the preprocessor from processing the same contents more than once:

```
#ifndef LIBFT_H
# define LIBFT_H

# include <stdlib.h>
# include <unistd.h>

size_t  ft_strlen(const char *s);

#endif
```

- `#ifndef` checks whether `LIBFT_H` has not yet been defined;
- `#define` defines this symbol to indicate that the contents have already been included;
- `#endif` ends the conditional block.

On subsequent inclusions, the guarded contents are simply skipped.

1.2 Automating Compilation with a `Makefile`

1.2.1 Why Use a Makefile?

A project like Libft contains dozens of `.c` files. Compiling them individually would be repetitive and inconvenient.

The **Makefile** automates this process through rules, dependencies, and commands run by `make`. It also lets you **rebuild only what has changed**.

1.2.2 Required Rules

Target	Purpose
`$(NAME)`	Builds the `libft.a` library.
`all`	Builds the library (default target).
`clean`	Removes the `.o` object files.
`fclean`	Removes the `.o` files and `libft.a`.
`re`	Cleans the project and rebuilds everything.
`bonus`	Adds the bonus functions to the library.

1.2.3 The *No Relink* Rule

`make` uses dependencies and file modification times to determine whether recompilation is needed.

Principle: if no required file has changed, running `make` again should neither recompile object files nor needlessly recreate the archive.

```
make        # First build
make        # Nothing to rebuild
make re     # Full rebuild
```

This rule avoids unnecessary operations and is one of the project requirements.

1.3 Compiling and Creating a Static Library

1.3.1 Step 1 — From Source File (`.c`) to Object File (`.o`)

The `cc` compiler transforms each source file into an **object file**, which contains machine code not yet linked into an executable.

```
cc -Wall -Wextra -Werror -c ft_strlen.c -o ft_strlen.o
```

Option meanings:

- `-Wall` enables a common set of warnings;
- `-Wextra` enables additional warnings;
- `-Werror` treats warnings as errors;
- `-c` compiles without linking;
- `-o` sets the name of the output file.

The first three flags are required for Libft.

1.3.2 Step 2 — Creating the Archive (`libft.a`)

A static `.a` library is an archive containing multiple `.o` object files. It is created with `ar`:

```
ar rcs libft.a ft_strlen.o ft_memset.o ft_memcpy.o
```

- `r` (***replace***) adds or replaces object files in the archive;
- `c` (***create***) suppresses the warning when creating a new archive;
- `s` (***symbol index***) generates the symbol index required for linking.

The result is `libft.a`, a library that can be reused in other C programs.

Important: the project specification requires `ar` to create this archive; using `libtool` is forbidden.

1.4 Coding Rules, Scope, and Memory Management

1.4.1 Global Variables Are Forbidden

A global variable is declared outside functions and has a lifetime that spans the program. In Libft, **global variables are forbidden**.

Functions must therefore work with their parameters, local variables, and dynamically allocated memory when needed.

1.4.2 The `static` Keyword and Helper Functions

Some functions, such as `ft_split`, may require several internal operations. To keep code readable, they can be divided into **helper functions**:

```
static size_t  ft_count_words(const char *s, char c);
```

The `static` keyword, applied to a function defined at file scope, limits its visibility to that `.c` file. Therefore:

- the function cannot be called from another file;
- it is not part of the public interface of `libft.a`;
- it avoids name conflicts with other functions.

1.4.3 Dynamic Allocation and Memory Leaks

The `malloc` function reserves memory on the heap. This memory remains allocated until it is released with `free`.

```
char    *str;

str = malloc(20);
if (!str)
    return (NULL);

/* Use str */

free(str);
```

Every successful allocation must be freed when it is no longer needed, either by the function itself or by the code that receives its result.

Important case: allocation failure. In `ft_split`, each word is allocated separately. If an allocation fails after earlier allocations succeeded, the function must:

1. free all words already allocated;
2. free the array that held their pointers;
3. return `NULL`.

Goal: ensure correct memory management, without memory leaks or double frees.

2 Function Classification and Fundamental C Concepts

Before reimplementing the Libft functions, it is essential to understand the C language mechanisms on which they rely.

Functions are grouped by similarity in behavior and implementation logic. This organization lets you study a concept and then apply it directly to several functions.

2.1 Function Families

2.1.1 Group 1 — Character Tests and Conversions

Functions: `ft_isalpha`,`ft_isdigit`,`ft_isalnum`,`ft_isascii`,`ft_isprint`
`ft_toupper`,`ft_tolower`.

These functions operate on a character represented by an integer (`int c`). They rely on ASCII numeric values, character comparisons, and conversions between uppercase and lowercase letters.

Main concept: numeric representation of characters.

2.1.2 Group 2 — Raw Memory Manipulation

Functions: `ft_memset`, `ft_bzero`, `ft_memcpy`, `ft_memmove`, `ft_memchr`, `ft_memcmp`.

These functions operate directly on memory regions of a known size, regardless of the type of data stored there. They rely on generic `void *` pointers, byte-by-byte reading and writing, and handling overlapping memory regions.

Main concept: pointers and memory representation.

2.1.3 Group 3 — String Analysis and Search

Functions: `ft_strlen`, `ft_strchr`, `ft_strrchr`, `ft_strncmp`, `ft_strnstr`, `ft_atoi`.

These functions traverse null-terminated strings to measure, search, compare, or convert their contents.

Main concept: C strings, indexing, and pointer arithmetic.

2.1.4 Group 4 — Bounded Copying and Concatenation

Functions: `ft_strlcpy`, `ft_strlcat`.

These functions copy or concatenate strings while respecting the capacity of the destination buffer.

Main concept: size management, buffer overflow prevention, and string termination.

2.1.5 Group 5 — Dynamic Allocation and String Creation

Functions: `ft_calloc`, `ft_strdup`, `ft_substr`, `ft_strjoin`, `ft_strtrim`, `ft_split`, `ft_itoa`.

These functions produce data stored in new memory regions, usually obtained with `malloc`.

Main concept: dynamic allocation, memory ownership, and error handling.

2.1.6 Group 6 — Iteration with Function Pointers

Functions: `ft_strmapi`, `ft_striteri`.

These functions apply an operation passed as an argument to each character in a string.

Main concept: function pointers and callbacks.

2.1.7 Group 7 — Writing to File Descriptors

Functions: `ft_putchar_fd`, `ft_putstr_fd`, `ft_putendl_fd`, `ft_putnbr_fd`.

These functions write characters, strings, or numbers using `write`.

Main concept: file descriptors and low-level input/output.

2.1.8 Group 8 — Linked Lists (Bonus)

Functions: `ft_lstnew`,`ft_lstadd_front`,`ft_lstsize`,`ft_lstlast`,`ft_lstadd_back` `ft_lstdelone`,`ft_lstclear`,`ft_lstiter`,`ft_lstmap`.

These functions are used to build and manipulate lists of nodes linked by pointers.

Main concept: self-referential structures, dynamic allocation, and double pointers.

2.2 Fundamental Concepts

2.2.1 Concept 1 — ASCII and Character Representation

In C, a character is represented by an integer value. The **ASCII** table maps numbers to common characters and defines 128 values, from `0` to `127`.

Character	ASCII value
'0'	48
'9'	57
'A'	65
'Z'	90
'a'	97
'z'	122

A `char` occupies one byte in the C sense; on common machines used for Libft, this byte usually contains 8 bits. Depending on the implementation, `char` may be signed or unsigned.

In code:

```
int ft_isdigit(int c)
{
    return (c >= '0' && c <= '9');
}
```

The compiler compares numeric values here. The characters `'0'` and `'9'` correspond to ASCII codes 48 and 57.

To convert a lowercase letter to uppercase:

```
if (c >= 'a' && c <= 'z')
    c -= ('a' - 'A');
```

Remember: standard library character-classification functions accept an `int`, allowing them to represent values of an `unsigned char` as well as the special value `EOF`.

2.2.2 Concept 2 — Generic Pointers and Memory Access

A **pointer** is a variable that holds the memory address of some data. A regular pointer has a type that tells the compiler how to interpret the data and how far to advance in an operation such as `p++`:

```
int  *p;
char *s;
```

By contrast, `void *` is a **generic pointer**: it represents an address without specifying the pointed-to object's type. In standard C, a `void *` cannot be dereferenced directly or used in pointer arithmetic.

To manipulate memory byte by byte, cast it to `unsigned char *`:

```
unsigned char *p;

p = (unsigned char *)buffer;
p[0] = 65;
p[1] = 66;
```

Each access then addresses one C byte. On a machine with 8-bit bytes, an `unsigned char` represents values from 0 to 255.

Application: `ft_memmove`. When two memory regions overlap, the copy direction matters.

```
Initial memory:   A B C D E
Copy 4 bytes from s to s + 1

Naive copy:       A A A A A
Expected result:  A A B C D
```

If the destination begins inside the source, after its start, copying forward destroys data that is still needed. In that case, copy **from the end to the beginning**.

Remember: `memcpy` does not guarantee a correct result when regions overlap; `memmove` handles this case.

2.2.3 Concept 3 — C Strings, Null Terminators, and Pointer Arithmetic

A C string is a sequence of characters whose end is marked by a null byte, written as `'\0'`.

```
char s[] = "1337";
```

In memory, it contains four visible characters and a fifth byte for the null terminator:

```
Index:    0    1    2    3     4
Value:   '1'  '3'  '3'  '7'  '\0'
```

Unlike some other languages, a C string does not automatically store its length. To determine it, the string must be traversed:

```
size_t i;

i = 0;
while (s[i] != '\0')
    i++;
```

There are two ways to access an element:

```
s[i]       Array indexing
*(s + i)   Pointer arithmetic
```

They are equivalent when the access is valid.

Application: `ft_strlcpy` and `ft_strlcat`. Distinguish between:

- the **buffer capacity**: space available for characters and the null terminator;
- the **string length**: number of characters before the null terminator;
- the **theoretical length**: length of the string we intended to create.

`ft_strlcpy` returns the source length. `ft_strlcat` normally returns the initial destination length plus the source length, or `dstsize + strlen(src)` if the destination has no null terminator in its first `dstsize` bytes.

Note: null termination is guaranteed only when the size conditions allow it, notably when `dstsize > 0` for `strlcpy`.

2.2.4 Concept 4 — Dynamic Allocation and Memory Management

In C, two common storage areas are often distinguished, as typically organized by common implementations:

- the **stack**, generally used for local variables and function-call management;
- the **heap**, used for dynamic allocations whose lifetime is controlled by the program.

`malloc` reserves a number of bytes and returns a pointer to the allocated region:

```
char *s;

s = malloc(10 * sizeof(char));
if (!s)
    return (NULL);
```

Memory allocated by `malloc` is uninitialized. After use, it must be released with `free`:

```
free(s);
```

A successful allocation that is never released when it is no longer needed causes a **memory leak**.

Application: `ft_calloc`. `calloc` allocates memory and initializes all bytes to zero. A robust implementation must also check that `count * size` does not exceed the capacity of `size_t`, to avoid integer overflow.

Application: `ft_split`. This function performs several allocations:

```
char **result
    |
    +--> result[0] --> "hello"
    +--> result[1] --> "1337"
    +--> result[2] --> "world"
    +--> result[3] --> NULL
```

The array and each word have their own allocation. If allocating a word fails, the previously allocated words must be freed, followed by the array.

Remember: every allocation needs a clear release strategy, including when execution fails.

2.2.5 Concept 5 — Function Pointers and Callbacks

In C, a function can be passed as an argument to another function using a **function pointer**. This separates two responsibilities: the main function controls data traversal, while the function passed as an argument defines the operation to perform.

```
char ft_example(unsigned int i, char c)
{
    (void)i;
    return (c + 1);
}
```

The prototype of `ft_strmapi` is:

```
char *ft_strmapi(char const *s,
                 char (*f)(unsigned int, char));
```

The parameter `f` is a pointer to a function that takes an index and a character, then returns a character. At each iteration, it can be called as follows:

```
result[i] = f(i, s[i]);
```

Important difference:

- `ft_strmapi` creates a new string from the characters returned by `f`;
- `ft_striteri` modifies the existing string by passing the address of each character to `f`.

This distinction corresponds to passing a character **by value** (`char`) or **by address** (`char *`).

2.2.6 Concept 6 — File Descriptors and `write`

On Unix-like systems, a program uses **file descriptors** to access open streams. A descriptor is an integer that identifies a file or stream opened by the process.

FD	Stream
0	`stdin` — Standard input
1	`stdout` — Standard output
2	`stderr` — Standard error

`write` writes bytes directly to a file descriptor:

```
write(1, "Hello", 5);
```

Here, `1` is the destination (`stdout`), `"Hello"` is the address of the data, and `5` is the requested number of bytes. `write` returns the number of bytes actually written, or `-1` on error. A partial write is possible.

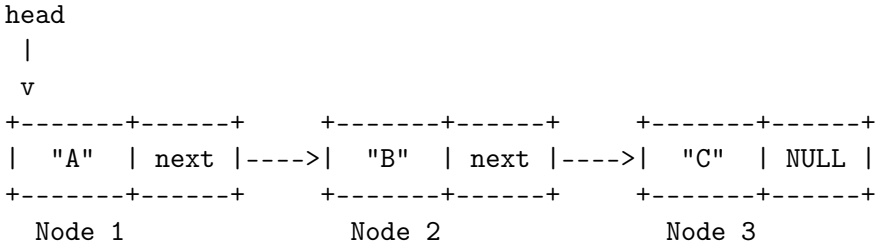
Application: `ft_putnbr_fd`**.** To print an integer, its digits must be converted to characters. The main operations are `% 10` to extract the last digit, `/ 10` to remove it, and `+ '0'` to convert a digit from 0 to 9 into its ASCII character.

Recursion makes it possible to write the digits in the correct order. Negative values must also be handled, especially `INT_MIN`, which cannot be made positive using a plain `int`.

2.2.7 Concept 7 — Self-Referential Structures and Linked Lists

A **linked list** is a dynamic data structure made up of elements called **nodes**. Unlike array elements, nodes are not necessarily stored at consecutive memory addresses.

Each node contains data and a pointer to the next node. The last node has `next = NULL`, marking the end of the list:



The variable `head` contains the address of the first node. Starting from this address, the list can be traversed by following the `next` pointers.

The `t_list` Structure. In Libft, the structure is defined as follows:

```
typedef struct s_list
{
    void            *content;
    struct s_list   *next;
}   t_list;
```

`content` is a generic pointer that can refer to different data types. `struct s_list *next` points to another structure of the same type, making it a **self-referential structure**. Finally, `typedef` creates the alias `t_list`, so `struct s_list` need not be written every time.

```
t_list *node;
```

This declaration creates a pointer to a node, but does not allocate a node.

Creating a Node with `malloc`. Nodes are generally allocated on the heap so they can exist independently of the function that creates them. Here is an example implementation of `ft_lstnew`:

```
t_list *ft_lstnew(void *content)
{
    t_list *node;

    node = malloc(sizeof(t_list));
    if (!node)
        return (NULL);
    node->content = content;
    node->next = NULL;
    return (node);
}
```

The function allocates enough space for the structure, checks that allocation succeeded, assigns the content to the node, initializes the next link to `NULL`, and returns the node address.

Important: `ft_lstnew` allocates the node but does not copy the data pointed to by `content`.

Accessing Members with `->`. Use the `.` operator to access a structure member. When you have a pointer to the structure, use `->`:

```
node->content;
(*node).content;
```

These two expressions are equivalent. The `->` operator combines pointer dereferencing and member access.

Traversing a Linked List. To traverse a list, start at its first node and follow the `next` addresses in sequence:

```
void print_list(t_list *lst)
{
    while (lst != NULL)
    {
        printf("%s\\n", (char *)lst->content);
        lst = lst->next;
    }
}
```

At each iteration, `lst->content` accesses the data and `lst = lst->next` advances to the next node. The loop stops when `lst == NULL`. In this example, the cast assumes the contents are C strings, which is not guaranteed for every list.

Complexity: traversing a list of `n` elements takes `O(n)` operations.

Understanding Double Pointers (`t_list **`). A `t_list *` pointer contains the address of a node. A `t_list **` double pointer contains the address of a pointer to a node:

```
t_list  *head;
t_list **ptr;

head = NULL;
ptr = &head;
```

Expression	Meaning
`ptr`	Address of the `head` variable.
`*ptr`	Value of `head`, i.e. the address of the first node.
`**ptr`	The first node structure, if the pointers are valid.

In C, arguments are passed by value. If a function receives only `t_list *head`, changing that local pointer does not change the caller's `head` variable. To change the head, pass its address using `&head`.

```
void ft_lstadd_front(t_list **lst, t_list *new)
{
    if (!lst || !new)
        return ;
    new->next = *lst;
    *lst = new;
}
```

The first assignment links the new node to the old head. The second makes the new node the head. The double pointer therefore lets the function modify a pointer owned by its caller.

Deletion and Memory Management. Every node created with `malloc` must be freed when it is no longer needed. Distinguish the memory for the node (`t_list`) from the memory for the data pointed to by `content`. In Libft, `ft_lstdelone` receives a function pointer `del`, responsible for freeing or cleaning up the content:

```
void ft_lstdelone(t_list *lst, void (*del)(void *))
{
    if (!lst || !del)
        return ;
    del(lst->content);
    free(lst);
}
```

`del(lst->content)` cleans up the data according to the provided function, then `free(lst)` releases the node.

Note: do not call `free` on data that was not dynamically allocated, such as a string literal.

To delete an entire list, `ft_lstclear` must save the address of the next node before freeing the current node:

```
void ft_lstclear(t_list **lst, void (*del)(void *))
```

```
{
    t_list *next;

    if (!lst || !del)
        return ;
    while (*lst)
    {
        next = (*lst)->next;
        ft_lstdelone(*lst, del);
        *lst = next;
    }
}
```

The order of operations is essential: accessing `next` after `free` would access released memory (a *use-after-free*). At the end, `*lst` is `NULL`.

The Nine Libft Bonus Functions.

Function	Purpose
`ft_lstnew`	Creates a new node.
`ft_lstadd_front`	Adds a node to the beginning of the list.
`ft_lstsize`	Counts the number of nodes.
`ft_lstlast`	Returns the last node.
`ft_lstadd_back`	Adds a node to the end.
`ft_lstdelone`	Deletes a node and its content using `del`.
`ft_lstclear`	Deletes all nodes starting from the given head.
`ft_lstiter`	Applies a function to the content of each node.
`ft_lstmap`	Creates a new list by transforming each content value.

The difference between `ft_lstiter` and `ft_lstmap` is important. `ft_lstiter` traverses existing nodes and applies a function to their contents without creating a new list. `ft_lstmap` builds a new list: for each original node, the function `f` produces the new node's content.

If an allocation fails during `ft_lstmap`, all newly created nodes and their contents must be cleaned up with `del`, without destroying the original list.

Complexity and Properties of Linked Lists.

Operation	Complexity
Access the first node	`O(1)`
Add to the beginning	`O(1)`
Traverse the list	`O(n)`
Find the last node	`O(n)`
Add to the end without a `tail` pointer	`O(n)`
Search for an element	`O(n)`

A linked list can grow and shrink dynamically. Adding or deleting a node can be fast when the necessary pointers are available. However, unlike an array, it does not provide direct access to the element at index `i`. Each node also requires extra memory for the `next` pointer.

Key points to remember:

1. a linked list is a sequence of nodes linked by pointers;
2. `head` holds the address of the first node;
3. `next` provides access to the next node, and `NULL` marks the end;
4. the `->` operator accesses structure members through a pointer;
5. `t_list **` lets a function modify the head pointer;
6. each allocated node must be freed correctly, along with its content when that content is owned by the list;
7. unlike arrays, linked lists do not guarantee direct access to elements by index.

Libft goal: know how to create a node, link nodes together, traverse a list, modify its head, and correctly free all memory in use.

2.3 Summary — Concepts to Master

Concept	Related functions
ASCII and characters	`ft_is*`, `ft_to*`
Pointers and raw memory	`ft_mem*`, `ft_bzero`
Strings and null termination	`ft_str*`, `ft_atoi`
Allocation and deallocation	`ft_calloc`, `ft_strdup`, `ft_split`, etc.
Function pointers	`ft_strmapi`, `ft_striteri`
File descriptors and writing	`ft_put*_fd`
Structures and double pointers	`ft_lst*`

These concepts form the technical foundation of the Libft project. Mastering them helps not only with reimplementing the required functions, but also with understanding the mechanisms used by C libraries.

3 Complete Reference to the 43 Libft Functions

This chapter is a **technical quick reference** to consult while implementing and testing the Libft project. Each function is presented with its prototype, purpose, return value, edge cases, and, where applicable, the permitted external functions.

Organization:

- Part 1: 23 libc functions;
- Part 2: 11 additional functions;
- Part 3: 9 linked-list bonus functions.

Convention: the "External functions" column lists the external functions permitted by the project subject, not reusable internal `ft_` functions.

3.1 Part 1 — Libc Functions (23 Functions)

3.1.1 A. Character Tests and Conversions

Function and prototype	Purpose	Return	Edge cases
`int ft_isalpha(int c)`	Tests whether `c` is an ASCII letter (A–Z, a–z).	`int`: nonzero if true, 0 otherwise.	Values outside the alphabetic ranges: 0.
`int ft_isdigit(int c)`	Tests whether `c` is a digit (0–9).	`int`: nonzero if true, 0 otherwise.	Non-digit values: 0.
`int ft_isalnum(int c)`	Tests whether `c` is alphanumeric.	`int`: nonzero if true, 0 otherwise.	Other characters: 0.
`int ft_isascii(int c)`	Checks whether `c` belongs to ASCII.	`int`: nonzero if `0 <= c <= 127`.	Negative values or values greater than 127: 0.
`int ft_isprint(int c)`	Tests whether `c` is printable ASCII.	`int`: nonzero for 32–126.	Control characters: 0.
`int ft_toupper(int c)`	Converts a lowercase ASCII letter to uppercase.	Converted character or `c`.	Non-lowercase character: unchanged.
`int ft_tolower(int c)`	Converts an uppercase ASCII letter to lowercase.	Converted character or `c`.	Non-uppercase character: unchanged.

Permitted external functions: None.

Remember: standard `<ctype.h>` functions expect `EOF` or a value representable as `unsigned char`. In Libft, these tests are usually implemented directly using ASCII ranges.

3.1.2 B. Raw Memory Manipulation

Function and prototype	Purpose	Return	Edge cases
`void *ft_memset(void *b, int c, size_t len)`	Fills `len` bytes with `c` converted to `unsigned char`.	`b`.	A valid region of sufficient size is required when `len > 0`.
`void ft_bzero(void *s, size_t n)`	Sets the first `n` bytes to zero.	`void`.	`n == 0`: no writes.
`void *ft_memcpy(void *dst, const void *src, size_t n)`	Copies `n` bytes between non-overlapping regions.	`dst`.	Overlap: undefined behavior.
`void *ft_memmove(void *dst, const void *src, size_t len)`	Copies `len` bytes, including when regions overlap.	`dst`.	Choose the copy direction to preserve the source.
`void *ft_memchr(const void *s, int c, size_t n)`	Searches for a byte in the first `n` bytes.	Address of the byte or `NULL`.	`n == 0`: `NULL`.
`int ft_memcmp(const void *s1, const void *s2, size_t n)`	Compares two memory regions over `n` bytes.	Negative, zero, or positive.	`n == 0`: 0.

Permitted external functions: None.

Important points:

- Use `unsigned char *` to access bytes.
- For `memcpy`, the source and destination must not overlap.
- For `memmove`, copy backward if the destination begins inside the source, after its start.
- For `memcmp`, compare values as `unsigned char`.
- A `NULL` pointer must not be dereferenced. Special tests involving `NULL` are not behavior guaranteed by libc.

3.1.3 C. String Analysis and Manipulation

Function and prototype	Purpose	Return	Edge cases
`size_t ft_strlen(const char *s)`	Counts characters before the null terminator.	String length.	`s == NULL`: undefined behavior.

`size_t ft_strlcpy(char *dst, const char *src, size_t dstsize)`	Copies `src`, limiting writes to the capacity of `dst`.	`strlen(src)`.	`dstsize == 0`: no copying.
`size_t ft_strlcat(char *dst, const char *src, size_t dstsize)`	Appends `src` to `dst` within the `dstsize` limit.	Theoretical concatenated length.	If `dst` has no null terminator within `dstsize`, returns `dstsize + strlen(src)`.
`char *ft_strchr(const char *s, int c)`	Searches for the first occurrence of `c`.	Pointer to the match or `NULL`.	Can search for the null terminator.
`char *ft_strrchr(const char *s, int c)`	Searches for the last occurrence of `c`.	Pointer to the match or `NULL`.	Can search for the null terminator.
`int ft_strncmp(const char *s1, const char *s2, size_t n)`	Compares at most `n` characters.	Negative, zero, or positive.	`n == 0`: 0; stops at the null terminator.
`char *ft_strnstr(const char *big, const char *little, size_t len)`	Searches for `needle` within the first `len` characters of `haystack`.	Address of the match or `NULL`.	Empty `needle`: returns `haystack`.
`int ft_atoi(const char *str)`	Converts the beginning of a string to an integer.	Resulting integer.	Leading whitespace, sign, no digits, and overflow.

Permitted external functions: None.

Important points:

- `ft_strlen` never counts the null terminator.
- `ft_strlcpy` and `ft_strlcat` must avoid buffer overflows and terminate the result if capacity allows.
- `ft_strncmp` stops at the first differing character, at the null terminator, or after `n` comparisons.
- `ft_strnstr` must find the entire `needle` within the `len` limit.
- `ft_atoi` skips leading whitespace, accepts an optional sign, and stops at the first non-digit. Overflow of an `int` has no guaranteed result under the standard `atoi` specification.

3.1.4 D. Allocation and Duplication

Function and prototype	Purpose	Return	Edge cases	External functions

`void *ft_calloc(size_t count, size_t size)`	Allocates `count * size` bytes initialized to zero.	Pointer or `NULL`.	Check for multiplication overflow; zero size: behavior is implementation-dependent.	`malloc`
`char *ft_strdup(const char *s1)`	Allocates a copy of the string `s1`.	New string or `NULL`.	Allocate `strlen(s1) + 1` bytes and handle allocation failure.	`malloc`

Remember: `calloc` initializes all bytes to zero, unlike `malloc`. A zero-size allocation may return `NULL` or a pointer that can be freed, depending on the implementation; it does not guarantee a non-null pointer.

3.2 Part 2 — Additional Functions (11 Functions)

3.2.1 A. String Creation and Transformation

Function and prototype	Purpose	Return	Edge cases	External functions
`char *ft_substr(char const *s, unsigned int start, size_t len)`	Extracts up to `len` characters starting at `start`.	Allocated string or `NULL`.	If `start` is beyond the string length, returns an allocated empty string.	`malloc`
`char *ft_strjoin(char const *s1, char const *s2)`	Concatenates two strings in a new memory block.	New string or `NULL`.	Check the total size and handle allocation failure.	`malloc`
`char *ft_strtrim(char const *s1, char const *set)`	Removes characters belonging to `set` from both ends.	New string or `NULL`.	If all characters are removed: returns an allocated empty string.	`malloc`
`char **ft_split(char const *s, char c)`	Splits a string using `c` as the delimiter.	Null-terminated array of strings.	Handle consecutive delimiters and clean up partial allocations.	`malloc`, `free`
`char *ft_itoa(int n)`	Converts an integer to a decimal string.	Allocated string or `NULL`.	Handle zero, negative values, and `INT_MIN`.	`malloc`

Important points:

- All produced strings must end with a null terminator.
- `ft_split` must also terminate its pointer array with `NULL`.
- In `ft_strtrim`, the characters in `set` form a set of individual characters, not a complete substring to search for.

- `ft_itoa` must reserve space for a possible minus sign.
- For functions that allocate memory, return `NULL` on failure and avoid memory leaks.

3.2.2 B. Iteration with Function Pointers

Function and prototype	Purpose	Return	Edge cases	External functions
`char *ft_strmapi(char const *s, char (*f)(unsigned int, char))`	Creates a string by applying `f` to each character.	New string or `NULL`.	Check allocation; handle invalid arguments carefully.	`malloc`
`void ft_striteri(char *s, void (*f)(unsigned int, char *))`	Applies `f` to each character of the existing string.	`void`.	`s` and `f` must be valid.	None

Key difference:

- `ft_strmapi` passes the character **by value** and builds a new string;
- `ft_striteri` passes its **address**, allowing the original string to be modified directly.

3.2.3 C. Writing to File Descriptors

Function and prototype	Purpose	Return	Edge cases	External functions
`void ft_putchar_fd(char c, int fd)`	Writes a character to `fd`.	`void`.	Invalid descriptor or write error.	`write`
`void ft_putstr_fd(char *s, int fd)`	Writes a string to `fd`.	`void`.	Invalid string or write error.	`write`
`void ft_putendl_fd(char *s, int fd)`	Writes a string followed by a newline.	`void`.	Invalid string or write error.	`write`
`void ft_putnbr_fd(int n, int fd)`	Writes the decimal representation of an integer.	`void`.	Negative values, zero, `INT_MIN`.	`write`

Important points:

- `write(fd, buf, count)` requests that `count` bytes be written to descriptor `fd`.
- `write` can fail or write fewer bytes than requested.
- `ft_putnbr_fd` can be implemented using recursion or digit conversion.
- These functions return `void`; they do not directly report a possible write failure.

3.3 Part 3 — Linked Lists (9 Bonus Functions)

3.3.1 Structure Definition

```
typedef struct s_list
{
    void            *content;
    struct s_list   *next;
}   t_list;
```

Each node contains a pointer to its data and a pointer to the next node.

3.3.2 Reference Table

Function and prototype	Purpose	Return	Edge cases	External functions
`t_list *ft_lstnew(void *content)`	Allocates a node, initializes `content`, and sets `next = NULL`.	Node or `NULL`.	Allocation failure; does not copy `content`.	`malloc`
`void ft_lstadd_front(t_list **lst, t_list *new)`	Inserts a node at the beginning.	`void`.	Initially empty list; invalid pointers.	None
`int ft_lstsize(t_list *lst)`	Counts the nodes in the list.	Number of nodes.	Empty list: 0.	None
`t_list *ft_lstlast(t_list *lst)`	Finds the last node.	Last node or `NULL`.	Empty list: `NULL`.	None
`void ft_lstadd_back(t_list **lst, t_list *new)`	Adds a node to the end of the list.	`void`.	Empty list: `new` becomes the head.	None
`void ft_lstdelone(t_list *lst, void (*del)(void *))`	Cleans up the content using `del`, then frees the node.	`void`.	Must not delete subsequent nodes.	`free`
`void ft_lstclear(t_list **lst, void (*del)(void *))`	Deletes a node and all its successors.	`void`.	After deletion, `*lst = NULL`.	`free`
`void ft_lstiter(t_list *lst, void (*f)(void *))`	Applies `f` to the content of each node.	`void`.	Empty list: no operation.	None
`t_list *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *))`	Creates a new list by transforming contents with `f`.	New head or `NULL`.	Clean up the new list if allocation fails.	`malloc`, `free`

3.3.3 Critical Bonus Considerations

Modifying the head. Functions using `t_list **lst` can modify the head pointer of the original list.

Safe deletion. Before freeing a node, save the address of the next node to avoid accessing memory that has already been freed.

Content ownership. The `content` pointer does not necessarily refer to an independent allocation. The `del` function defines how the data is cleaned up.

Handling `ft_lstmap` errors. If creating a node fails, free the newly produced content if necessary, then clean up all nodes already created with `ft_lstclear`. The original list must not be destroyed.

3.4 Final Validation Checklist

- ☐ All 43 prototypes exactly match the project subject.
- ☐ The `libft.h` file contains all required declarations.
- ☐ The functions cause no out-of-bounds memory access with valid arguments.
- ☐ Returned strings are properly terminated with `'\0'`.
- ☐ Allocations are checked and freed correctly.
- ☐ Memory functions handle overlapping regions correctly when overlap is allowed.
- ☐ Edge cases (zero, empty string, `INT_MIN`, empty list) are tested.
- ☐ Bonus functions do not lose any nodes or allocations.
- ☐ Compilation passes with `-Wall -Wextra -Werror`.
- ☐ The Makefile follows the required rules and avoids unnecessary rebuilds.
- ☐ The 42 Norm is respected.

Note: this reference describes expected behavior for valid inputs and relevant edge cases. For behavior that libc leaves undefined (invalid pointers, `atoi` overflow, etc.), do not invent a guaranteed return value.

End of the Libft Reference Guide

This guide brings together the project's technical framework, fundamental C concepts, and a reference for the 43 functions to implement. It serves as a resource for development, testing, and evaluation of the first Common Core 1337 project.